

Stepwise

A teammate's guide to the DSA practice app: what it does, how its pieces connect, and how to work on it safely.

The 20-second model

A single-page HTML/CSS/JavaScript app talks to a small Python standard-library HTTP server. The server handles accounts, email, local test runners, certificates, and writes progress to a local SQLite database.

FRONT END	SERVER	DATA	RUNNERS
index.html HTML + CSS + browser JavaScript	server.py Python 3 stdlib HTTP + API	data/loop.sqlite3 SQLite, created on first run	Browser iframe: JavaScript Local Python / Java: problem and algorithm tests

Project snapshot

Current repository: github.com/nikhilkottakota2-dotcom/stepwise (private). Local app address: **http://127.0.0.1:8000**. The project is a local prototype; its Python and Java execution endpoints are not a public-safe sandbox.

Generated from the checked-in app files on 2026-10-02. This is a code-oriented map of the implementation, not a production deployment runbook.

Read this guide

This PDF has clickable section links and PDF outline bookmarks. Click a section below, or open the bookmarks/outline panel in your PDF reader.

[1. Product and current scope](#)

[2. Architecture and request flow](#)

[3. Accounts, data, and email](#)

[4. Languages and test execution](#)

[5. DSA curriculum and roadmap](#)

[6. Run locally and configure](#)

[7. Working agreement and production boundary](#)

Start here when contributing

Launch the app, create or sign in to an account, run one problem and one algorithm test, then inspect the API route and matching SQLite table before changing that flow.

Repository map

Path	Responsibility
index.html	UI markup, styling, navigation, roadmap/catalog, browser test harnesses, auth UI, certificates.
server.py	HTTP server, REST-like API, SQLite schema, email/token flows, local Python/Java test execution.
start_stepwise.ps1	Windows launcher: temporary SMTP environment, secure prompt for the mail app password, then starts server.py.
README.md	Setup, environment variables, current runner limitations, and deployment warnings.
.gitignore	Excludes data/ and local secrets/artifacts from Git; database contents are per machine.

1. Product and current scope

Stepwise separates two learning loops that share one account and progress store:

Mode	User path	Record / reward
Problem solving	Pick one of six starter challenges, choose a language, edit the solution, run test cases.	A passing result can be recorded to the account. Solving all six earns the problem-solving certificate.
Algorithm practice	Pick a level and algorithm, follow the guided visualization/steps, edit a solution, run tests.	A passing recorded session saves the algorithm and awards a printable completion certificate.

What is implemented today

- Overview dashboard, problem set, algorithm lab, DSA roadmap, progress data, and certificate view.
- Email-verified account creation, login/logout, resend verification, forgot password, and password reset.
- Per-account language preference, solved-problem history, algorithm completions, and certificate records.
- Problem workspaces and algorithm workspaces with a larger editor area and an explicit Run tests action.

UI items labelled Saved problems, Notes, or general progress are currently navigation/placeholders unless a related API/schema is present. Do not promise persistence for those without implementing it.

Catalog counts

The visible problem catalog has 6 problems. The algorithm lab contains 26 guided algorithms: 9 beginner, 11 intermediate, and 6 advanced. The navigation label's '128' is a UI display count, not 128 implemented problem records.

Product success path

1. Learn	2. Implement	3. Verify	4. Persist
Read a problem or inspect the algorithm walk-through.	Choose JS, Python, Java, C, or C++ template and write code.	Run supplied cases; all cases must pass to show success.	If signed in, Stepwise records the result and may issue a certificate.

2. Architecture and request flow

The browser is the presentation layer; the Python process is both static-file server and application API. No separate frontend build or dependency install is required.

Browser / index.html	Python / server.py	SQLite / data/loop.sqlite3
Renders UI; owns problem/algorithm catalogs and guided UI state. JavaScript runs in a hidden sandboxed iframe. Sends JSON requests to /api/ endpoints.	ThreadedHTTPServer + SimpleHTTPRequestHandler. Validates account requests, checks session cookies, sends SMTP mail, invokes local Python/Java subprocess runners, stores progress.	Local single-file database. Tables use foreign keys; user data is keyed by user_id. Created automatically by initialize() at startup.

Typical test-to-progress sequence

Step	Connection / behavior
1 - Select	UI chooses a problem or algorithm and a language; the starter code is loaded in the editor.
2 - Execute	JavaScript: page sends source/cases to a hidden iframe. Python/Java: browser POSTs code to server API; server writes a temporary harness and launches local interpreter/compiler.
3 - Decide	A test report returns. Failure/time-out stops the path. Passing all configured tests enables the record action.
4 - Save	Authenticated browser POSTs /api/solved or /api/algorithm-complete. Server inserts progress/certificate records into SQLite.
5 - Refresh	GET /api/me returns the signed-in profile and saved progress/certificates for the UI.

API surface

HTTP route	Purpose
GET /api/me	Current account, solved problems, algorithm completions, certificates.
GET /api/verify?token=...	Redeem email verification token.
POST /api/register, /api/login, /api/logout	Account and session lifecycle.
POST /api/resend-verification, /api/forgot-password, /api/reset-password	Email token lifecycle.
POST /api/run-tests, /api/run-algorithm-tests	Execute configured language-specific test harness.
POST /api/solved, /api/algorithm-complete, /api/profile	Persist progress, certificates, and profile preference.

3. Accounts, data, and email

SQLite schema (created by initialize() in server.py)

Table	Stored data / relationships
users	Name, unique case-insensitive email, PBKDF2 password hash + salt, language, email_verified, created timestamp.
email_verification_tokens	Hashed one-time token, user foreign key, expiry (24 hours).
password_reset_tokens	Hashed one-time token, user foreign key, expiry (1 hour).
sessions	Hashed session token, user foreign key, expiry (30 days). Cookie is HttpOnly and SameSite=Strict.
solved_problems	Unique user/problem pair and completion time.
algorithm_completions	Unique user/algorithm pair, level, completion time.
certificates	Track and title per user, issue time; unique user/track/title.

Identity and email flow

- Registration validates email/name and requires a 10-character password; it creates a salted PBKDF2-HMAC-SHA256 hash (310,000 rounds).
- Email verification and reset tokens are random, stored as SHA-256 hashes, expire, and are removed after use or replacement.
- Login requires verified email. Successful login sets a 30-day random session cookie; only its hash is persisted. Reset revokes active sessions.
- Email is sent using Python smtplib with STARTTLS on port 587 or implicit TLS on 465. SMTP settings are environment variables, not code constants, except the local launcher preselects a developer sender address.

Secret handling

Never put SMTP_PASSWORD, OAuth/PAT values, or real user data in this guide, Git, screenshots, logs, or a deployment config committed to the repository. Local launcher prompts without echo and places the value in that process environment. Rotate the previously shared development mail app password before any public launch.

Local data is not cloud sync

The current account database lives at data/loop.sqlite3. Git ignores data/ so real account records should not be pushed. Backups, migration scripts, multi-instance storage, and data retention policy are not included in this prototype.

4. Languages and test execution

Language choice in the editor does not mean every language has an execution harness. Treat this as the actual support matrix in the current code:

Language	Problem set (6)	Algorithm lab (26)	Where it runs / dependency
JavaScript	Supported; supplied cases	Supported; supplied cases	Browser hidden iframe with a 2.5 s timeout and a restrictive iframe CSP. No server subprocess.
Python	Supported; per-problem harness	Supported; per-algorithm fixtures	Local server launches Python in isolated mode; 3 s execution timeout. Python 3 required.
Java	Supported; per-problem harness	Supported; per-algorithm fixtures	Local server compiles with javac and runs java; JDK required; compile timeout 15 s, run timeout 3 s.
C	Template only; harness not configured	Template only; execution not configured	Problem runner checks for gcc, then still reports harness unconfigured. Algorithm runner not configured.
C++	Template only; harness not configured	Template only; execution not configured	Problem runner checks for g++; harness still unconfigured. Algorithm runner not configured.

Test counts and result semantics

- Each of the six problem suites has 2 or 3 test cases. Exact inputs and expected values are in `PROBLEM_TESTS` in `server.py` and `problemSpecs` in `index.html`.
- Each algorithm currently has a single curated fixture/check per language family; JavaScript cases are in `algoTestCases`, Python/Java fixtures in `ALGORITHM_CASES`.
- A green/pass status means all configured checks returned expected values. It does not prove performance, hidden-case correctness, memory safety, or production-grade isolation.
- Progress should only be saved after a successful run and an authenticated request. A guided walk-through completion by itself is not the same as passing tests in every UI path; preserve and inspect the current route before changing the rule.

Important execution boundary

The Python and Java runners execute user-submitted code as child processes on the same machine as the app. Short timeouts and temporary folders are not a hardened sandbox. Bind to localhost; do not expose `/api/run-tests` or `/api/run-algorithm-tests` to the public internet.

5. DSA curriculum and roadmap

The roadmap is a learning sequence, not a guarantee of prerequisite enforcement. The algorithm lab's selector groups these lessons by level:

Level	Topics in the current catalog
Beginner (9)	Linear Search; Binary Search; Bubble Sort; Selection Sort; Insertion Sort; Two Pointers; Stack; Queue; Hash Map Lookup.
Intermediate (11)	Merge Sort; Quick Sort; Sliding Window; Linked List Reversal; Tree DFS; Tree BFS; BST Search; Heap / Priority Queue; Backtracking; Topological Sort; Union Find.
Advanced (6)	Dijkstra; Bellman-Ford; Kruskal MST; Dynamic Programming - Knapsack; Longest Common Subsequence; Trie.

A sensible teammate learning path

Phase	Build intuition around	Look for
1. Foundations	Arrays, linear/binary search, sorting, hash maps, stack and queue.	Index boundaries, invariants, stable/unstable behavior, lookup tradeoffs.
2. Patterns	Two pointers, sliding windows, recursion, merge/quick sort, backtracking.	What state moves each step; where repeated work is avoided.
3. Structures	Linked lists, trees, BST, heaps, tries, union-find.	Shape of the structure, operation costs, traversal order.
4. Graphs + DP	DFS/BFS, topological sort, Dijkstra, Bellman-Ford, Kruskal, knapsack, LCS.	Constraints that select an algorithm; state, transition, and proof.

Certificate behavior

An algorithm certificate is issued for an individual completed algorithm title. The problem-solving certificate is issued only after all six catalog problems have been recorded. Certificates are generated in a printable browser window; they are not digitally signed credentials.

Current algorithm completion gap

The Run tests success path saves completion after passing. A separate Complete session button can also save after the guided steps without proving tests passed, and the API does not validate test evidence. If the product rule is 'tests must pass before recording', unify those paths and enforce it on the server.

For each new lesson, keep the concept, step visualization, starter signatures, JavaScript cases, Python fixture, and Java fixture aligned. The lesson selector, implementation maps, and test cases are maintained in different sections/files, so coverage drift is possible.

6. Run locally and configure

Windows local start

1	2	3
Install Python 3.10+.	From the project folder run .start_stepwise.ps1; enter the Gmail app password at the hidden prompt.	Open http://127.0.0.1:8000 and create an account. Verify the email before signing in.

Environment contract

Variable	What it controls	Safe value pattern
SMTP_HOST	SMTP server hostname	smtp.example.com
SMTP_PORT	587 STARTTLS or 465 SMTP over TLS	587
SMTP_USER	Mailbox login name	development or production mailbox
SMTP_PASSWORD	Provider app password / SMTP secret	Set only in process environment or secret manager; never commit.
SMTP_FROM	Sender display identity	Stepwise
APP_BASE_URL	Host embedded in verification/reset links	Local: http://127.0.0.1:8000 ; production: deployed HTTPS URL.
PORT	HTTP listen port	Defaults to 8000.
C_COMPILER / CXX_COMPILER	Optional C/C++ executable lookup	gcc / g++; test harnesses are still incomplete.

Connections map

- Browser -> local Python HTTP server: same-origin HTTP, JSON API calls, session cookie.
- Python server -> SQLite file: direct sqlite3 connection; foreign keys enabled for each connection.
- Python server -> SMTP provider: outbound TLS connection for verification and password-reset messages.
- Python server -> local Python/JDK: subprocess execution in temporary directories for non-JavaScript tests.
- Browser -> Google Fonts: remote stylesheet import for DM Sans, DM Mono, and Manrope; app should still have system fallbacks if unavailable.

Local DB reset (destructive)

Deleting data/loop.sqlite3 erases local accounts and progress. Back it up before troubleshooting or moving machines. The DB is intentionally excluded from Git.

7. Working agreement and production boundary

Safe teammate habits

- Keep the app bound to 127.0.0.1 while the code-execution APIs use local subprocesses.
- Never commit data/loop.sqlite3, SMTP secrets, reset/verification links, or real account data. Check git status before pushing.
- When changing an algorithm, update the curriculum record, starter code, JavaScript case, and Python/Java fixture as one unit.
- When changing an account flow, follow it end-to-end: UI, API route, token table/session cookie, database mutation, and user-visible response.
- When changing certificates/progress, check idempotency (unique keys), logged-out behavior, and account refresh (/api/me).
- Keep UI language honest: distinguish available templates from executable language support; distinguish local prototype behavior from deployed behavior.

Before a public launch

Area	Required next work
Hosting	Use a production WSGI/ASGI server and managed host; do not expose the development HTTP server directly.
Code execution	Remove arbitrary local subprocess execution or run it in a hardened isolated worker/container with resource and network restrictions. Authenticate and rate-limit execution endpoints.
Transport + cookies	Serve HTTPS, add Secure cookie flag and origin/CSRF protections, review headers, rate limits, and session rotation.
Email	Replace the temporary personal mailbox and app password; use a verified sending domain, provider secrets store, SPF/DKIM/DMARC, and production APP_BASE_URL.
Database	Move to managed persistent storage, define migrations/backups/restore, retention, and privacy controls.
Operations	Add structured safe logs, monitoring, dependency/runtime pinning, error handling, and a deployment rollback path.
Verification	Run an agreed smoke/security test plan in the target hosting environment; the local prototype is not proof of public readiness.

Current-state summary

Stepwise is a working local DSA learning prototype with accounts, email verification/reset hooks, per-user progress, 26 guided algorithms, six starter problems, and partial multi-language test execution. Its core architecture is simple; its execution backend and single-machine SQLite setup are the main constraints to resolve before publishing.

Code references

Open the private repository: [Stepwise on GitHub](#). Primary implementation references: index.html, server.py, start_stepwise.ps1, README.md, and .gitignore.

End of teammate guide